BASICS OF NUMBER THEORY

Some Notation

$\mathbb{Z} = \{\ldots, -1, 0, 1, \ldots\}$ —integers

$\mathbb{Z}_+ = \{1, 2, \ldots\}$ —positive integers

$\mathbb{N} = \{0, 1, 2, \ldots\}$ —natural numbers

$d|b-d$ divides b $(5|10)$

For $a, N \in \mathbb{Z}$ let $\gcd(a, N)$ be the largest $d \in \mathbb{Z}_+$ such that $d|a$ and $d|N$.

Example: $\gcd(20; 90) = 10$.

Some Notation

- For $N \in \mathbb{Z}_+$ let

$$\mathbb{Z}_N = \{0, 1, 2, \ldots, N-1\}$$

$$\mathbb{Z}_N^* = \{i \in \mathbb{Z}_N : \gcd(i, N) = 1\}$$

$$\varphi(N) = |\mathbb{Z}_N^*|$$

- **Example:** $N = 12$

$$\mathbb{Z}_{12} =$$

$$\mathbb{Z}_{12}^* =$$

$$\varphi(12) =$$

Division and Mod

- **Fact.** For any $a \in \mathbb{Z}, N \in \mathbb{Z}_+$ there are unique $r, q \in \mathbb{Z}$ s.t.

  $$a = Nq + r$$

  $$0 \leq r < N$$

- q is called the **quotient** and r is called the **remainder**.
- $a \bmod N = r \in \mathbb{Z}_N$ is the remainder when a is divided by N.
- **Def.** $a \equiv b (\bmod N)$ iff $(a \bmod N) = (b \bmod N)$
- **Example.** $a = 17$ and $N = 3$ $q =$ $r =$

  $$17 \bmod 3 =$$

  $$17 \equiv 14 (\bmod 3)$$

Groups

- **Def.** Let G be a non-empty set and let $\cdot$ denote a binary operation on G. We say that G is a group if it has the following properties:

  1. **Closure:** For every $a, b \in G$ it is the case that $a \cdot b$ is also in G.
  2. **Associativity:** For every $a, b, c \in G$ it is the case that $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
  3. **Identity:** There exists an element $1 \in G$ such that $a \cdot 1 = 1 \cdot a = a$ for all $a \in G$.
  4. **Invertibility:** For every $a \in G$ there exists a unique $b \in G$ such that $a \cdot b = b \cdot a = 1$.

inverse, denoted a^{-1}

Group Examples

- For $N \in \mathbb{Z}_+$
  - $\mathbb{Z}_N$ — group under addition modulo N

Group Examples

- For $N \in \mathbb{Z}_+$
  - $\mathbb{Z}_N^*$ — group under multiplication modulo N

- In any group, we can define an exponentiation operation:
  - If $i = 0$ then a^i is defined to be 1
  - If $i > 0$ then $a^i = a \cdot a \cdots a$ (i times)
  - If $i < 0$ then $a^i = a^{-1} \cdot a^{-1} \cdots a^{-1}$ ($j = -i$ times)

- For all $a \in G$ and all $i, j \in \mathbb{Z}$:
  - $a^{i+j} = a^i \cdot a^j$

    $(a^i)^j = a^{ij}$

    $a^{-i} = (a^i)^{-1} = (a^{-1})^i$

- The **order** of a group is its size.
- **Fact.** Let G be a group and let $m = |G|$ be its order.

  Then $a^m = 1$ for all $a \in G$.
- **Fact.** Let G be a group and let $m = |G|$ be its order.

  Then $a^i = a^{i \bmod m}$ for all $a \in G$ and all $i \in \mathbb{Z}$.
- **Example.** Let us work in the group $\mathbb{Z}_{21}^* = \{1, 2, 4, 5, 8, 10, 11, 13, 16, 17, 19, 20\}$ under the operation of multiplication modulo 21

  The order is ...

  $5^{86} \bmod 21 =$

- **Def.** If G is a group, a set $S \subseteq G$ is called a subgroup if it is a group in its own right, under the same operation as that under which G is a group.

- If we already know that G is a group, there is a simple way to test whether S is a subgroup:
  - it is one if and only if $x \cdot y^{-1} \in S$ for all $x, y \in S$. Here y^{-1} is the inverse of y in G.

- **Fact.** Fact. Let G be a group and let S be a subgroup of G Then the order of S divides the order of G.

- **Example.** Let $G = \mathbb{Z}_{11}$, $S = \{1, 3, 4, 5, 9\}$ is a subgroup.

MEASURING RUNNING TIMES OF ALGORITHMS DEALING WITH NUMBERS

Algorithms and Their Running Times

- Since in cryptography we will be working with

  BIG numbers (typical sizes 2^{512}, 2^{1024}, 2^{2048}), the complexity of algorithms taking numbers as inputs is measured as **a function of the bit-length of the numbers**.

- $|7| =$

- $|2^{1024}| =$

- E.g. PrintinBinary(A), where $A = 2^k$ takes k operations

Some Basic Algorithms

Algorithm	Input		Output	RunningTime
INT-DIV	a, N	$(N > 0)$	(q, r) with $a = Nq + r$ and $0 \leq r < N$	$\mathcal{O}(\|a\| \cdot \|N\|)$
MOD	a, N	$(N > 0)$	$a \bmod N$	$\mathcal{O}(\|a\| \cdot \|N\|)$
EXT-GCD	a, b	$((a, b) \neq (0, 0))$	$(d, \bar{a}, \bar{b})$ with $d = \gcd(a, b) = a\bar{a} + b\bar{b}$	$\mathcal{O}(\|a\| \cdot \|b\|)$
MOD-ADD	a, b, N	$(a, b \in \mathbb{Z}_N)$	$(a + b) \bmod N$	$\mathcal{O}(\|N\|)$
MOD-MULT	a, b, N	$(a, b \in \mathbb{Z}_N)$	$ab \bmod N$	$\mathcal{O}(\|N\|^2)$
MOD-INV	a, N	$(a \in \mathbb{Z}_N^*)$	$b \in \mathbb{Z}_N^*$ with $ab \equiv 1 (\mathrm{mod} N)$	$\mathcal{O}(\|N\|^2)$
MOD-EXP	a, n, N	$(a \in \mathbb{Z}_N)$	$a^n \bmod N$	$O\left(\|n\| \cdot \|N\|^2\right)$
EXP_G	a, n	$(a \in G)$	$a^n \in G$	$2\|n\|G$-operations

Addition

- $(a, b) \mapsto a + b$

$$\begin{array}{cccccccc} & 1 & 0 & 1 & 1 & 0 & 1 \\ + & & & 1 & 0 & 1 & 1 \\ \hline & 1 & 1 & 1 & 0 & 0 & 0 \end{array}$$

- By the usual "carry" algorithm, we can compute $a + b$ in time $\mathcal{O}(|a| + |b|)$.

- Addition is **linear** time.

Multiplication

- $(a, b) \mapsto ab$

$$
\begin{array}{cccccccc}
 & & & 1 & 0 & 1 & 1 & 1 & 0 \\
 & & \times & & & & 1 & 0 & 1 \\
\hline
 & & & 1 & 0 & 1 & 1 & 1 & 0 \\
 & & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
+ & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\
\hline
 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0
\end{array}
$$

- By the usual algorithm, we can compute ab in time $\mathcal{O}(|a| \cdot |b|)$
- Multiplication is **quadratic** time.

Integer Division

- $\text{INT-DIV}(a, N)$ returns (q, r) such that

$$a = qN + r$$
$$0 \leq r < N$$

- **Example:** $\text{INT-DIV}(17, 3) = (5, 2)$
- By the usual algorithm, we can compute $\text{INT-DIV}(a, N)$ in time $\mathcal{O}(|a| \cdot |N|)$.
- Integer division is **quadratic** time.

MOD

- $(a, N) \mapsto a \bmod N$
- But

$$(q, r) \leftarrow \text{INT-DIV}(a, N)$$
$$\text{return } r$$

  computes $a \bmod N$, so again the time needed is $\mathcal{O}(|a| \cdot |N|)$.

- Mod is **quadratic** time.

About GCD

- **Fact.** If $a, N \in \mathbb{Z}$ and $(a, N) \neq (0, 0)$ then $\gcd(a, N)$ is the smallest positive integer in the set

$$\{a \cdot a' + N \cdot N' : a', N' \in \mathbb{Z}\}$$

- **Corollary.** If $d = \gcd(a, N)$ then there are "weights" $a', N' \in \mathbb{Z}$ such that

$$d = a \cdot a' + N \cdot N'$$

- **Example.** $\gcd(20, 12) = 4$ and $4 = 20 \cdot a' + 12 \cdot N'$ for

$$a' =$$

$$N' =$$

Extended GCD

- $\text{EXT-GCD}(a, N) \mapsto (d, a', N')$ such that

$$d = \gcd(a, N) = a \cdot a' + N \cdot N'.$$

- **Fact.** Let $(q, r) = \text{INT-DIV}(a, N)$. Then, $\gcd(a, N) = \gcd(N, r)$.

- **Example.** $\text{INT-DIV}(17, 3) = (5, 2)$ so $\gcd(17, 3) = \gcd(3, 2)$.

Extended GCD

- Running time analysis is non-trivial and shows that the time is $\mathcal{O}(|a| \cdot |N|)$.
- So the extended gcd can be computed in **quadratic** time.

Modular Inverse

- For a, N such that $\gcd(a, N) = 1$, we want to compute $a^{-1} \bmod N$ meaning the unique $a' \in \mathbb{Z}_N^*$ satisfying $aa' \equiv 1 (\bmod N)$.

  But if we let $(d, a', N') \leftarrow \text{EXT-GCD}(a, N)$ then

  $$d = 1 = \gcd(a, N) = a \cdot a' + N \cdot N'.$$

  But $N \cdot N' = 0 (\bmod N)$ so $aa' \equiv 1 (\bmod N)$.

  Algorithm MOD-INV(a, N)

  $(d, a', N') \leftarrow \text{EXT-GCD}(a, N)$

  return $a' \bmod N$

- Modular inverse can be computed in **quadratic** time.

Modular Exponentiation

- Let G be a group and $a^n \in G$. For $n \in \mathbb{N}$, we want to compute $a^n \in G$.

  We know that

  $$a^n = \underbrace{a \cdot a \cdots a}_{n}$$

  Consider

  $y \leftarrow 1$
  for $i = 1, \ldots, n$ do $y \leftarrow y \cdot a$
  return y

- **Question.** Is this a good algorithm?

Modular Exponentiation

- Let G be a group and $a^n \in G$. For $n \in \mathbb{N}$, we want to compute $a^n \in G$.

  We know that

  $$a^n = \underbrace{a \cdot a \cdots a}_{n}$$

  Consider

  $y \leftarrow 1$
  for $i = 1, \ldots, n$ do $y \leftarrow y \cdot a$
  return y

- **Question.** Is this a good algorithm?

- **Answer.** It is correct but **VERY SLOW**. The number of group operation is

  $$\mathcal{O}(n) = \mathcal{O}(2^{|n|})$$

so it is exponential time. For $n \approx 2^{512}$ it is prohibitively expensive.

Fast Exponentiation Idea

- We can compute

$$a \longrightarrow a^2 \longrightarrow a^4 \longrightarrow a^8 \longrightarrow a^{16} \longrightarrow a^{32}$$

  in just 5 steps by repeated squaring. So we can compute a^n in i steps when $n = 2^i$.

- But what if n is not a power of 2?

Fast Exponentiation Example

- Suppose the binary length of n is 5, meaning the binary representation n of has the form $b_4 b_3 b_2 b_1 b_0$

$$\begin{aligned} n &= 2^4 b_4 + 2^3 b_3 + 2^2 b_2 + 2^1 b_1 + 2^0 b_0 \\ &= 16 b_4 + 8 b_3 + 4 b_2 + 2 b_1 + b_0 \ . \end{aligned}$$

- We want to compute a^n. Our exponentiation algorithm will proceed to compute the values $y_5, y_4, y_3, y_2, y_1, y_0$ in turn, as follows:

$$\begin{aligned} y_5 &= 1 \\ y_4 &= y_5^2 \cdot a^{b_4} = a^{b_4} \\ y_3 &= y_4^2 \cdot a_3^{b_3} = a^{2b_4 + b_3} \\ y_2 &= y_3^2 \cdot a^{b_2} = a^{4b_4 + 2b_3 + b_2} \\ y_1 &= y_2^2 \cdot a^{b_1} = a^{8b_4 + 4b_3 + 2b_2 + b_1} \\ y_0 &= y_1^2 \cdot a^{b_0} = a^{16b_4 + 8b_3 + 4b_2 + 2b_1 + b_0} \ . \end{aligned}$$

Fast Exponentiation

- Let $\text{bin}(n) = b_{k-1} \dots b_0$ be the binary representation of n, meaning

$$n = \sum_{i=0}^{k-1} b_i 2^i$$

Algorithm $\text{EXP}_G(a, n)$ $\quad //a \in G, n \geq 1$

$b_{k-1} \dots b_0 \leftarrow \text{bin}(n)$

$y \leftarrow 1$

for $i = 1, \dots, k-1$ downto 0 do $y \leftarrow y^2 \cdot a^{b_i}$

return y

- The running time is $\mathcal{O}(|n|)$ group operations.
- $\text{MOD-EXP}(a, n, N)$ returns $a^n \bmod N$ in time $\mathcal{O}(|n| \cdot |N|^2)$, meaning is **cubic** time.

Algorithms Summary

Algorithm	Input	Output	Time
INT-DIV	a, N	q, r	quadratic
MOD	a, N	$a \bmod N$	quadratic
EXT-GCD	a, N	(d, a', N')	quadratic
MOD-ADD	a, b, N	$a + b \bmod N$	linear
MOD-MULT	a, b, N	$ab \bmod N$	quadratic
MOD-INV	a, N	$a^{-1} \bmod N$	quadratic
MOD-EXP	a, n, N	$a^n \bmod N$	cubic
EXP_G	a, n	$a^n \in G$	$\mathcal{O}(\lvert n \rvert)$ G-ops

What is Fast, What is Slow?

- We will want the schemes' algorithms to be efficient, i.e., their running time, which is a function of the security parameter should be
  - **polynomial** (in the security parameter)

- We will call an attack successful if the adversary is efficient, i.e., its resources, which are functions of the security parameter should be
  - **polynomial** (in the security parameter)

What is Fast, What is Slow?

- For security, we will want the advantage of any efficient adversary, which is a function of the security parameter to be
  - **negligible** (in the security parameter)

- **Def.** A function $g : \mathbb{Z}_+ \to \mathbb{R}$ is negligible if it vanishes faster than the reciprocal of any polynomial. (That is, for every positive integer c there is an integer n_c such that $g(n) \leq n^{-c}$ for all $n \geq n_c$.)

Order of a Group Element

- Let G be a (finte) group.

  Def: The **order** of g, denoted $o(g)$, is the smallest integer $n \geq 1$ such that $g^n = 1$.

- Why does the order exist? Since G is finite, the sequence

$$\mathbf{1} = g^0, g^1, g^2, \ldots$$

  must repeat, meaning there are i, j with $i < j$ and $g^i = g^j$. But then

$$\mathbf{1} = g^0 = g^{-i} g^i = g^j g^{-i} = g^{j-i}$$

  so there is some $m \geq 1$ such that $g^m = 1$.

Order Determination

- Let $G = \mathbb{Z}_{11}^* = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.

i	0	1	2	3	4	5	6	7	8	9	10
$2^i \bmod 11$	1	2	4	8	5	10	9	7	3	6	1
$5^i \bmod 11$	1	5	3	4	9	1	5	3	4	9	1

$o(2) =$

$o(5) =$

Subgroup Generated by g

- **Def:** For $g \in G$ we let

$$\langle g \rangle = \{g^0, g^1, \ldots, g^{o(g)-1}\} .$$

This is a subgroup of G and its order (that is, its size) is the order $o(g)$ of G.

Subgroup Order

- **Fact:** The order $|S|$ of a subgroup S always divides the order $|G|$ of the group G.
- **Fact:** The order $o(g)$ of $g \in G$ always divides $|G|$.
- **Example:** If $G = \mathbb{Z}_{11}^*$, then

$$|G| =$$

$$o(2) =$$

$$o(5) =$$

Subgroups Generated by a Group Element

- Let $G = \mathbb{Z}_{11}^* = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.

i	0	1	2	3	4	5	6	7	8	9	10
$2^i \bmod 11$	1	2	4	8	5	10	9	7	3	6	1
$5^i \bmod 11$	1	5	3	4	9	1	5	3	4	9	1

so

$$\langle 2 \rangle =$$

$$\langle 5 \rangle =$$

Generators

- **Def.** An element g of the group is called a **generator** of G if $\langle g \rangle = G$.
- **Fact.** g is a **generator** iff $o(g) = |G|$.
- **Def.** A group is cyclic if it contains a generator.

Generators

- Let $G = \mathbb{Z}_{11}^* = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.

i	0	1	2	3	4	5	6	7	8	9	10
$2^i \bmod 11$	1	2	4	8	5	10	9	7	3	6	1
$5^i \bmod 11$	1	5	3	4	9	1	5	3	4	9	1

So

$$\langle 2 \rangle = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$$
$$\langle 5 \rangle = \{1, 3, 4, 5, 9\}$$

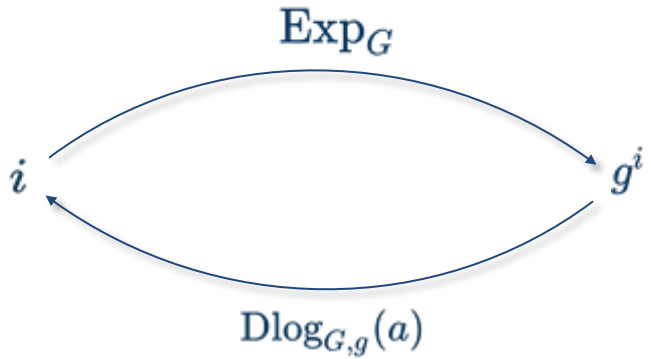
  - Is 2 a generator?
    YES because $\langle 2 \rangle = \mathbb{Z}_{11}^*$.
  - Is 5 a generator?
    NO because $\langle 5 \rangle \neq \mathbb{Z}_{11}^*$.
  - Is $\mathbb{Z}_{11}^*$ cyclic?
    YES because it has a generator.

Discrete Log

- If $G = \langle g \rangle$ is cyclic then for every $a \in G$ there is a **unique** exponent $i \in \{0, \ldots, |G| - 1\}$ such that $g^i = a$. We call i the discrete logarithm of a to base g and denote it by

$$\mathrm{Dlog}_{G,g}(a)$$

- The discrete log function is the inverse of the exponentiation function



- Let $G = \mathbb{Z}_{11}^* = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$. We know that 2 is a generator, so $\mathrm{Dlog}_{G,2}(a)$ is the exponent $i \in \{0, \ldots, 9\}$ such that $2^i \equiv a \pmod{11}$.

i	0	1	2	3	4	5	6	7	8	9
$2^i \bmod 11$	1	2	4	8	5	10	9	7	3	6

a	1	2	3	4	5	6	7	8	9	10
$\mathrm{DLog}_{G,2}(a)$										

Finding Cyclic Groups

- **Fact 1:** Let p be a prime. Then $\mathbb{Z}_p^*$ is cyclic.
- **Fact 2:** Let G be any group whose order $m = |G|$ is a prime number. Then G is cyclic.
- **Note:** $|\mathbb{Z}_p^*| = p - 1$ is not prime, so **Fact 2** doesn't imply **Fact 1**!

Computing Discrete Log

- Let $G = \langle g \rangle$ be a cyclic group with generator $g \in G$.

  Input: $X \in G$

  Desired Output: $\mathrm{DLog}_{G,g}(X)$

  That is, we want x such that $g^x = X$.

  ```
  for x = 0, ..., |G| − 1 do
      X' ← g^x
      if X' = X then return x
  ```

  Is this a good algorithm?

Computing Discrete Log

- There is a better-than-exhaustive-search method to compute discrete logarithms, but its $\mathcal{O}(\sqrt{|G|})$ running time is still exponential and prohibitive.
  - Is there a faster algorithm?
  - Is there a polynomial time algorithm, meaning one with running time $\mathcal{O}(n^c)$ for some constant c where $n = \log |G|$?
- **State of the art.** There are faster algorithms in some groups, but no polynomial time algorithm is known.
- This (apparent, conjectured) computational intractability of the discrete log problem makes it the basis for cryptographic schemes in which breaking the scheme requires discrete log computation.

Computing Discrete Log in $\mathbb{Z}_p^*$

- Let p be a prime and $G = \mathbb{Z}_p^*$. Then there is an algorithm that finds discrete logs in G in time

$$e^{1.92(\ln p)^{1/3}(\ln \ln p)^{2/3}}$$

- This is not polynomial

- **Note:** The actual running time is $e^{1.92(\ln q)^{1/3}(\ln \ln q)^{2/3}}$ where q is the largest prime factor of $p-1$, but we chose p so that $q \approx p$, for example $p - 1 = 2q$ for q a prime.

Computing Discrete Log in Elliptic Curve Groups

- Let G be a prime-order group of points over an elliptic curve. Then the best known algorithm to compute discrete logs takes time

$$\mathcal{O}(\sqrt{p})$$

where $p = |G|$.

Comparison

- Say we want 80-bits of security, meaning discrete log computation by the best known algorithm should take time 2^{80}.
  - If we work in $\mathbb{Z}_p^*$, (p a prime) we need to set $|\mathbb{Z}_p^*| = p - 1 \approx 2^{1024}$.
  - But if we work on an elliptic curve group of prime order p then it suffices to set $p \approx 2^{160}$.

  Why?

$$e^{1.92(\ln 2^{1024})^{1/3}(\ln \ln 2^{1024})^{2/3}} \approx \sqrt{2^{160}} = 2^{80}$$

Why Smaller Groups Preferable?

Group Size	Cost of Exponentiation
2^{160}	1
2^{1024}	260

- Exponentiation takes time cubic in $\log|G|$ where G is the group.
- Encryption and decryption will be 260 times faster in the smaller group!

Constructing Cyclic Groups

- We will need to build (large) groups over which our cryptographic schemes can work, and find generators in these groups.
- How do we do this efficiently?

Finding Generators

- If $|G|$ is prime then every $g \in G - \{1\}$ is a generator.
- If $G = \mathbb{Z}_p^*$ where p is a prime
  - It may be hard in general to find a generator
  - But easy if the prime factorization of $p - 1$ is known

Pick and Check

repeat
$g \xleftarrow{\$} G - \{1\}$
until $(\text{TEST-GEN}_G(g) = \text{true})$

- How do we design TEST-GEN_G ?
- How many iterations does the algorithm take?

- We say that p is safe prime if $p - 1 = 2q$ for some prime q.
- **Example:** 7 is safe prime because $7 - 1 = 2(3)$ and 3 is a prime. We will address the above question for safe primes.

Generators Mod 7

- Let $G = \mathbb{Z}_7^* = \{1, 2, 3, 4, 5, 6\}$.

i	1	2	3	4	5	6
1^i	1	1	1	1	1	1
2^i	2	4	1	2	4	1
3^i	3	2	6	4	5	1
4^i	4	2	1	4	2	1
5^i	5	4	6	2	3	1
6^i	6	1	6	1	6	1

The generators are **3** and **5**

- We observe that g is a generator if and only if $g^2 \neq 1$ and $g^3 \neq 1$.

Testing Generators

- Suppose p is a safe prime, meaning $p - 1 = 2q$ for a prime q.
- **Fact:** $g \in \mathbb{Z}_p^*$ is a generator if and only if $g^2 \not\equiv 1$ and $g^q \not\equiv 1$ modulo p.
- **Example:** Let $p = 7$ so that $q = 3$. Then $g \in \mathbb{Z}_7^*$ is a generator if and only if $g^2 \not\equiv 1$ and $g^3 \not\equiv 1$ modulo 7.

How Many Generators?

- Suppose p is a safe prime, meaning $p - 1 = 2q$ for a prime q.
- **Fact:** $\mathbb{Z}_p^*$ has $q - 1$ generators.
- **Example:** Suppose $p = 7$ so that $q = 3$. Then $\mathbb{Z}_7^*$ has $q - 1 = 2$ generators.

  So if $g \xleftarrow{\$} G - \{1\}$ then

$$\Pr\left[\langle g \rangle = \mathbb{Z}_p^*\right] = \frac{q-1}{p-2} = \frac{q-1}{2q-1} \approx \frac{1}{2}$$

- **Example:** If $p = 7$ and $g \xleftarrow{\$} \mathbb{Z}_7^* - \{1\}$ then

$$\Pr\left[\langle g \rangle = \mathbb{Z}_7^*\right] = \frac{3-1}{7-2} = \frac{2}{5}$$

Finding Generators Mod Safe Primes

- Suppose p is a safe prime with $p - 1 = 2q$.

repeat
$g \xleftarrow{\$} G - \{1\}$
until $(g^2 \not\equiv 1 (\bmod p)$ and $g^q \not\equiv 1 (\bmod p))$

- The probability that a generator is found in a given step is

$$\frac{q-1}{2q-1} \approx \frac{1}{2}$$

so the expected number of iterations of the algorithm is about 2

Recall...

- We want to figure out how to find
  - A large safe prime p
  - A generator g of $\mathbb{Z}_p^*$

  so that we can work over $\mathbb{Z}_p^* = \langle g \rangle$.

- So far we solved the second problem. What about the first?

Finding Primes

- **Desired:** An efficient algorithm that given an integer k returns a prime $p \in \{2^{k-1}, \ldots, 2^k - 1\}$ such that $q = (p-1)/2$ is also prime.

Algorithm Findprime(k)
  do
    $p \xleftarrow{\$} \{2^{k-1}, \ldots, 2^k - 1\}$
  until (p is prime and $(p-1)/2$ is prime)
  return p

  - How do we test primality?
  - How many iterations do we need to succeed?

Naïve Primality Testing

- **Given:** integer N

  Output: **TRUE** if N is prime, **FALSE** otherwise.

$$\begin{aligned}&\text{for } i = 2, \ldots, \lceil \sqrt{N} \rceil \text{ do} \\ &\quad \text{if } N \bmod i = 0 \text{ then return false} \\ &\text{return true}\end{aligned}$$

- Correct but **SLOW**: Exponential running time. However, we have:
  - $\mathcal{O}(|N|^3)$ time randomized algorithms
  - Even a $\mathcal{O}(|N|^3)$ time deterministic algorithm

Density of Primes

- Let $\pi(N)$ be the number of primes in the range $1, \ldots, N$. So if $p \xleftarrow{\$} \{1, \ldots, N\}$ then

$$\Pr[p \text{ is a prime}] = \frac{\pi(N)}{N}$$

- **Fact:** $\pi(N) \sim \frac{N}{\ln(N)}$

  so

$$\Pr[p \text{ is a prime}] \sim \frac{1}{\ln(N)}$$

If $N = 2^{1024}$ this is about $0.001488 \approx 1/1000$.

So the number of iterations taken by our algorithms to find a prime is not too big.

Squares

- We say that a is a square (or quadratic residue) modulo p if there exists b such that $b^2 \equiv a \pmod{p}$.

  We let

$$J_p(a) = \begin{cases} 1 & \text{if } a \text{ is a square mod } p \\ 0 & \text{if } a \bmod p = 0 \\ -1 & \text{otherwise} \end{cases}$$

  be the Legendre or Jacobi symbol of a modulo p

  Let $p = 11$. Then

  - Is 4 a square modulo p?
    YES, because $2^2 \equiv 4 \pmod{11}$
  - Is 5 a square modulo p?

Set of Squares

- We let

$$\begin{aligned}\mathrm{QR}\left(\mathbb{Z}_p^*\right) &= \left\{a \in \mathbb{Z}_p^* : a \text{ is a square mod } p\right\} \\ &= \left\{a \in \mathbb{Z}_p^* : \exists b \in \mathbb{Z}_p^* \text{ such that } b^2 \equiv a (\bmod p)\right\}\end{aligned}$$

Example

- Let $p = 11$

a	1	2	3	4	5	6	7	8	9	10
$a^2 \bmod 11$	1	4	9	5	3	3	5	9	4	1

Then

$$\mathrm{QR}(\mathbb{Z}_{11}^*) = \{1, 3, 4, 5, 9\}$$

a	1	2	3	4	5	6	7	8	9	10
$J_{11}(a)$	1	-1	1	1	1	-1	-1	-1	1	-1

Observe

- There are 5 squares and 5 non-squares.
- Every square has exactly 2 square roots.

Relation to Discrete Log

- Recall that 2 is a generator of $\mathbb{Z}_{11}^*$

a	1	2	3	4	5	6	7	8	9	10
$\mathrm{DLog}_{\mathbb{Z}_{11}^*,2}(a)$	0	1	8	2	4	9	7	3	6	5
$J_{11}(a)$	1	-1	1	1	1	-1	-1	-1	1	-1

so

$$J_{11}(a) = 1 \text{ iff } \mathrm{DLog}_{\mathbb{Z}_{11}^*,2}(a) \text{ is even}$$

This makes sense because for any generator g.

$$g^{2j} = \left(g^j\right)^2$$

is always a square!

Relation to Discrete Log

- **Fact:** If $p \geq 3$ is a prime and g is a generator of $\mathbb{Z}_p^*$ then

$$\mathrm{QR}\left(\mathbb{Z}_p^*\right) = \{g^i : 0 \leq i \leq p-2 \text{ and } i \text{ is even}\}$$

- **Example:** if $p = 11$ and $g = 2$ then $p - 2 = 9$ and the squares are

$$2^0 \bmod 11 = 1$$
$$2^2 \bmod 11 = 4$$
$$2^4 \bmod 11 = 5$$
$$2^6 \bmod 11 = 9$$
$$2^8 \bmod 11 = 3$$

Computing J_p

- **Fact:** If $p \geq 3$ is a prime then for any a

$$J_p(a) \equiv a^{\frac{p-1}{2}} \pmod{p}$$

- **Example:** Let $p = 11$.
  - Let $a = 5$. We know that 5 is a square, meaning $J_{11}(5) = 1$. Now compute

$$a^{\frac{p-1}{2}} \equiv 5^5 \equiv (25)(25)(5) \equiv 3 \cdot 3 \cdot 5 \equiv 45 \equiv 1 \pmod{11}.$$

  - Let $a = 6$. We know that 6 is not a square, meaning $J_{11}(6) = -1$. Now compute

$$a^{\frac{p-1}{2}} \equiv 6^5 \equiv (36)(36)(6) \equiv 3 \cdot 3 \cdot 6 \equiv 54 \equiv -1 \pmod{11}.$$

Computing J_p

- **Fact:** If $p \geq 3$ is a prime then for any a

$$J_p(a) \equiv a^{\frac{p-1}{2}} \pmod{p}$$

This yields a cubic-time algorithm to compute the Legendre symbol, meaning determine whether or not a given number is a square:

Algorithm TEST-SQ(p, a)

$s \leftarrow a^{\frac{p-1}{2}} \bmod p$

if $s = 1$ then return 1 else return -1

- **Claim:** Let $p \geq 3$ be a prime and let g be a generator of $\mathbb{Z}_p^*$. Then $J_p\left(g^{xy} \bmod p\right) = 1$ if and only if $J_p\left(g^{x} \bmod p\right) = 1$ or $J_p\left(g^{y} \bmod p\right) = 1$ for all $x, y \in \mathbb{Z}_{p-1}$.

Multiplicity of Legendre Symbol

- **Fact:** If $p \geq 3$ is a prime then for any a, b

$$J_p(ab) = J_p(a) \cdot J_p(b)$$

- **Example:** Let $p = 11$.

a	1	2	3	4	5	6	7	8	9	10
$J_{11(a)}$	1	-1	1	1	1	-1	-1	-1	1	-1

a	b	ab	$J_{11}(a)$	$J_{11}(b)$	$J_{11}(ab)$	$J_{11}(a) \cdot J_{11}(b)$
5	6	8	1	-1	-1	-1
2	7	3	-1	-1	1	1

Inversion of Legendre symbol

- **Fact:** If $p \geq 3$ is a prime then for any $a \in \mathbb{Z}_p^*$

$$J_p\left(a^{-1}\right) = J_p(a)$$

- **Example:** $p = 11$

a	1	2	3	4	5	6	7	8	9	10
$J_{11(a)}$	1	-1	1	1	1	-1	-1	-1	1	-1

a	a^{-1}	$J_{11}(a)$	$J_{11}\left(a^{-1}\right)$
3	4	1	1
7	8	-1	-1

- **Example:** $\mathrm{QR}(\mathbb{Z}_{11}^*)$

a	1	2	3	4	5	6	7	8	9	10
$a^2 \bmod 11$	1	4	9	5	3	3	5	9	4	1

$$\mathrm{QR}(\mathbb{Z}_{11}^*) = \{1, 3, 4, 5, 9\}$$

- Recall that $\mathbb{Z}_{11}^*$ is cyclic and 2 is a generator.
- **Fact:** A generator is always a non-square. (But not all non-squares are generators).

a	1	2	3	4	5	6	7	8	9	10
$\mathrm{DLog}_{\mathbb{Z}_{11}^*,2}(a)$	0	1	8	2	4	9	7	3	6	5
$J_{11}(a)$	1	−1	1	1	1	−1	−1	−1	1	−1

- **Fact:** Let $p \geq 3$ be a prime and let g be a generator of $\mathbb{Z}_p^*$. Then $\mathrm{QR}(\mathbb{Z}_p^*) = \{g^i : i \in \mathbb{Z}_{p-1} \text{ and } i \text{ is even}\}$ and $|\mathrm{QR}(\mathbb{Z}_p^*)| = (p-1)/2$.